THE UNIVERSITY OF MELBOURNE

IT Capstone Project

COMP 30022 2025

Week 7 Software testing

Outline

- Testing principles
- Types of testing
- When and how
- Documenting testing

Introduction

- Software Testing:
  - *A process to identify the correctness, completeness and quality of developed computer software, which includes a set of activities conducted with the intent of finding errors in software so that it could be corrected before the product is released to the end users.*
- Testing is important!
- Software bugs could be expensive and even dangerous.

Seven Principles

(International Software Testing Qualifications Board)

- Exhaustive testing is not possible
- Defect clustering
- Pesticide paradox
- Testing shows presence of defects
- Absence of errors fallacy
- Early testing
- Testing is context dependent

Seven Principles

- Suppose you have 20 input fields to test, each having 5 possible values, the number of combinations to be tested would be 5^{20}!

- Principle 1: Exhaustive testing is not possible.

Seven Principles

Which operation is most likely to cause your Operating system to fail?

A. Open Microsoft Word

B. Open Chrome Browser

C. Open 10 heavy graphics applications simultaneously

Principle 2: Defect clustering - a small number of modules contain most of the defects detected.

Seven Principles (Cont'd)

- If the same tests are repeated over and over again, eventually the same test cases will no longer find new bugs.

- Principle 3: Pesticide paradox.

✓ The test cases need to be regularly reviewed & revised, adding new & different test cases to help find more defects.

Seven Principles (Cont'd)

- Difficult to argue that your software is bug-free.

- Principle 4: Testing shows a presence of defects.

Seven Principles (Cont'd)

- Software can be 99% bug-free but not meet the needs and requirements of the clients

- Principle 5: Absence of error is a fallacy

- Finding and fixing defects does not help if the system build is unusable and does not fulfill the user's needs and requirements

Seven Principles (Cont'd)

- Much cheaper to fix a defect in the early stages of testing.

- Principle 6: Early testing

Principle 7: Testing is context-dependent.

Concepts

- White box testing – testing your own code
- Black box testing – testing some one else's code without looking

- Test-driven development

Test Plan

- Document strategy, objectives, tools and processes
- Ensure:
  - All team members understand the objectives
  - Testing is not neglected throughout the project.
  - Forces the team to think about how they will verify the quality of their product

Test Plan

- Include
  - Testing objectives.
  - Testing tools/frameworks.
  - In scope/Out of scope (what aspects of the system will/won't be tested).
  - A clear description of what types of testing will be completed. Should provide detail as to who will be conducting the testing and when testing will occur.
  - Bug reporting strategy (when a bug is identified, what process should the team follow?).

Types of Testing

Type	Description
Unit testing	Checking whether each software unit performs as expected. The finest level of granularity in testing.
Integration testing	Checking whether multiple software components/units operate correctly.
Acceptance testing	Checking whether the application meets the client's requirements. Acceptance tests are created based on the user stories.

WHY DO UNIT TESTING?

Manual or automated?

- Manual testing is possible
  - Need to create documentation detailing step by step instructions, screenshots and comments showing success and failure scenarios
  - Should be tested and confirmed by a reviewer (not the person who wrote the code)
- Automated testing
  - Many tools available to simplify the process:
    - JUnit: a popular and free testing tool for Java.
    - Jest: a popular and free testing tool for JavaScript.
    - Testing library.
    - Mocha: a popular and free testing tool for JavaScript.

Integration Testing

- Individual units are combined and tested together as a group. Try to test the connection between the units.
- *Big Bang*: all or most of the units are combined together and tested at one go. This approach is taken when the testing team receives the entire software in a bundle.
- Top Down: top-level units are tested first and lower level units are tested step by step after that.
- *Bottom Up:* bottom level units are tested first and upper-level units step by step after that.
- *Sandwich/Hybrid* : a combination of Top Down and Bottom Up approaches.

- Some tools exist to help automate integration testing, e.g. Playwright, Cypress

Code Coverage

- Common metric for examining thoroughness of your test plan
- Looks at the proportion of your code covered by a test
- Not an indicator of success (e.g. 100% coverage doesn't mean your program is perfect). May test:
  - *Function coverage:* how many of the functions defined have been called.
  - *Statement coverage:* how many of the statements in the program have been executed.
  - *Branches coverage:* how many of the branches of the control structures (if statements for instance) have been executed.
  - *Condition coverage:* how many of the Boolean sub-expressions have been tested for a true and a false value.
  - *Line coverage:* how many of lines of source code have been tested.

```
[Sten-Pittets-Macbook-Pro:coverage spittet$ istanbul cover coverage-tutorial.js
 true
 ============================================================================
 Writing coverage object [/Users/spittet/Developer/Atlassian/tutorials/coverage/coverage/coverage.json]
 Writing coverage reports at [/Users/spittet/Developer/Atlassian/tutorials/coverage/coverage]
 ============================================================================

 =============================== Coverage summary ===============================
 Statements   : 83.33% ( 5/6 )
 Branches     : 50% ( 1/2 )
 Functions    : 100% ( 1/1 )
 Lines        : 83.33% ( 5/6 )
 ================================================================================
 Sten-Pittets-Macbook-Pro:coverage spittet$
```

(Image: Atlassian)

Tools like Cypress and Istanbul are helpful for automating

Acceptance Testing

- Makes sure the system meets the requirements specified by the client
- Acceptance criteria based on requirements, may be written before implementation
- For acceptance criteria:
  - May have 1-3 acceptance criteria per user story.
  - Acceptance criteria are typically written before implementation.
  - They are independently testable.
  - They have a defined pass or fail result.

Acceptance Criteria

User Story ID	User Story	Given	When	Then
1	As an athlete user, I can see the latest update for my sessions in my dashboard's calendar so that I can track my current program's completion in a more informative way.	I have been doing exercises everyday	I check sessions in my dashboard's calendar	• I can see completion of my current and previous programs
2	As an athlete user, I can receive some badges for my achievements after I overcome some challenges during my sessions so that I can be motivated to stay on track for my "Building".	Many challenges related to my personal program listed in my session	I complete challenges in my session's program	• I receive badges for corresponding challenges
		I have received badge when I complete the challenge	I click on the social media button next to the badge section	• It shares my badges on the social platform
3	As an athlete user, I can see route map tracking in my session description & feedback so that I can get an overview of session feedback visually.	I am doing some outdoor exercises	I click session in the dashboard	• It show route tracking map • feedback

Test cases

A documented set of preconditions (prerequisites), procedures (inputs / actions) and postconditions (expected results) that a tester uses to determine whether a system under test satisfies requirements or works correctly. A test case can have one or multiple test scripts and a collection of test cases is called a test suite.

Test cases – when to write them?

- Within Agile Sprints (in this subject, parallel to development)
- When an iterative approach is being applied to software development and work is contained in “sprints”, the definition of tasks performed for the sprint tasks might include writing test cases for each task in the sprint.

- Writing tests in parallel to the development of the task enhances team collaboration and quality to the final product. During the dev process, some scenarios may need clarification, or the UI might need to be adjusted and the requirements updated.

Test cases (templates)

US 01: As an admin,
I want to add new users to the system
so that they can login to it

TC 01: Add User (successful)

Test Type:	**Execution Type:**
Functional	Manual

Objective:

Verify if a user is correctly added.

Setup:

<List the pre-conditions to carry through this test case >

Pre-Conditions:

<List the conditions after the test execution >

Notes:

[1] Add user name

[2] Add user e-mail

[3] Submit the form

[4] Verify that the user is added to the list.

Do not use special chars, digits, maximum allowed number of chars (username: 20, name: 55, email: 55), to fill username, name and email fields.

Exit application and begin application using a new contact. Verify if user profile works properly.

[5] Add various users

** Application generates a random password for the new user and sends it by email.*

** Username, name and email are mandatory fields.*

** User name field allow the following especial characters: _ . (underscore and dot).*

** Name field allow the following especial characters: _ - (underscore and negative).*

** Email field allow the following especial characters: - + _ . ‘ (negative, positive, underscore, dot and apostrophe).*

** The email field is validated*

** The event will be logged in Business Log*

**User profiles:*

Admin

Student

Time constraint:

Minimum: 20 min

Maximum: 25 min

Confluence

Test cases (templates)

US 01: As an admin,
I want to add new users to the system
so that they can login to it

TC 02: Add User (unsuccessful)

Test Type:	**Execution Type:**
Functional	Manual

Objective:

Verify application behavior when user could not be added.

Setup:

<List the pre-conditions to carry through this test case >

•

Pre-Conditions:

<List the conditions after the test execution >

•

Notes:

[1] Start filling the form for a new user and cancel operation. User list is not modified.

[2] Try to add user with an existent username.

[3] Try to add user when username, name and/or email fields are empty.

[4] Try to add user with Invalid content type:

[4.1] Using more than maximum allowed number of chars (username: 20, name: 55, email: 55).

[4.2] Use special chars, accents, empty, spaces, only spaces to filled username, name and email fields.

[4.3] Try to add user when email format is not valid.

** User name field allow the following special characters: _ . (underscore and dot).*

** Name field allow the following special characters: _ - (underscore and negative).*

** Email field allow the following special characters: - + _ . ‘ (negative, positive, underscore, dot and apostrophe).*

** The email field is validated*

Time constraint:

Minimum: 20 min

Maximum: 25 min

Confluence

Linking user stories and test cases in the sprint

US01 → AC01, AC02

AC01 → TC01, TC02, TCn

Verification

Acceptance criteria ID	User Story	Result	Evidence (sequence of steps conducted)
AC01	CRM-117 (Register) As a customer (Lawrence, Cin, Fatima or Jordan) I want to register an account so that I have a place to organise my contacts.	Pass	1. User navigates to register page 2. User provides a previously unused email and two matching passwords 3. User clicks "sign up" and is redirected to the dashboard page, logged in to their new account

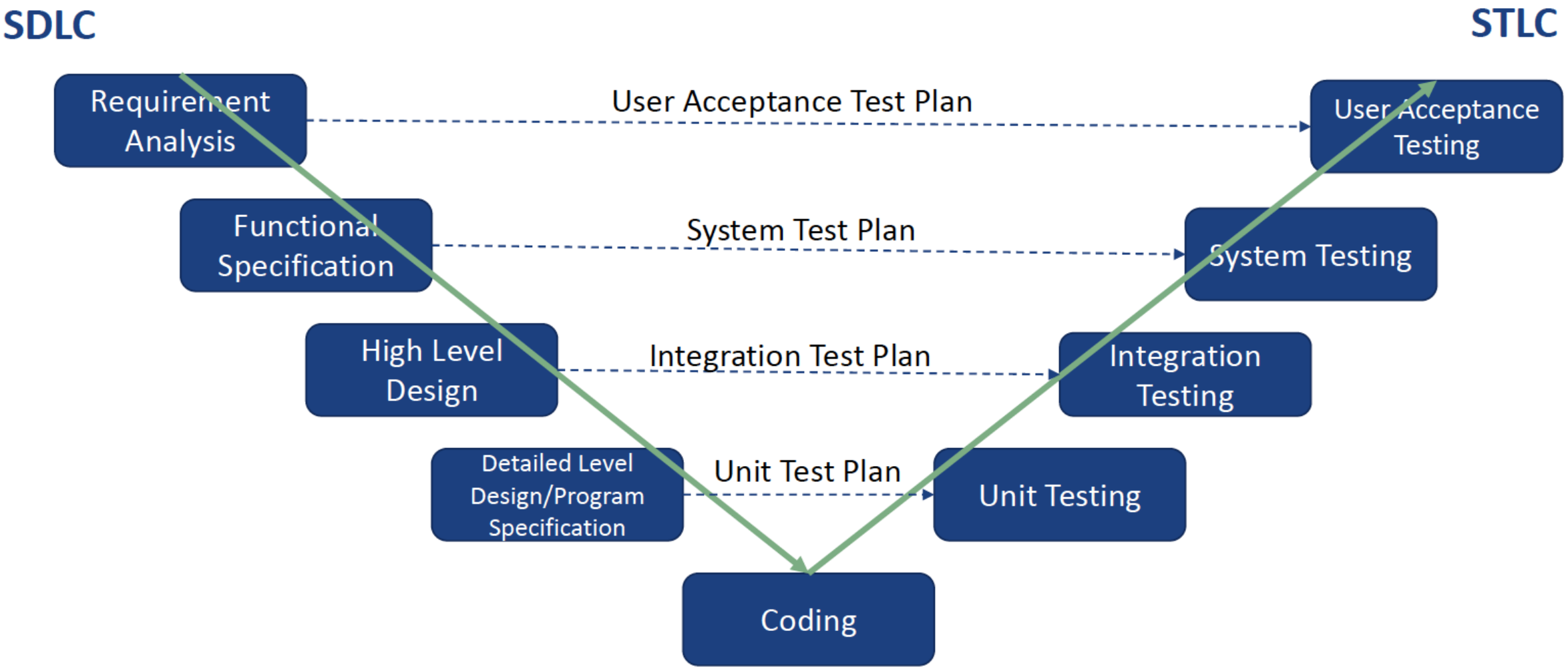
Load Testing

- How well your program can cope with processing a large amount of data or multiple of users using your server at the same time?
- It's a type of performance testing, which determines the performance of a program (not functional correctness of a program)
- E.g., 100 users querrying your server at the same time, would your program be slowing down, your server be crashed, etc?

User experience testing

- Weak point of student software projects
- Users will do different things than you expect
- Must be planned for
- Question: Who should conduct user experience testing

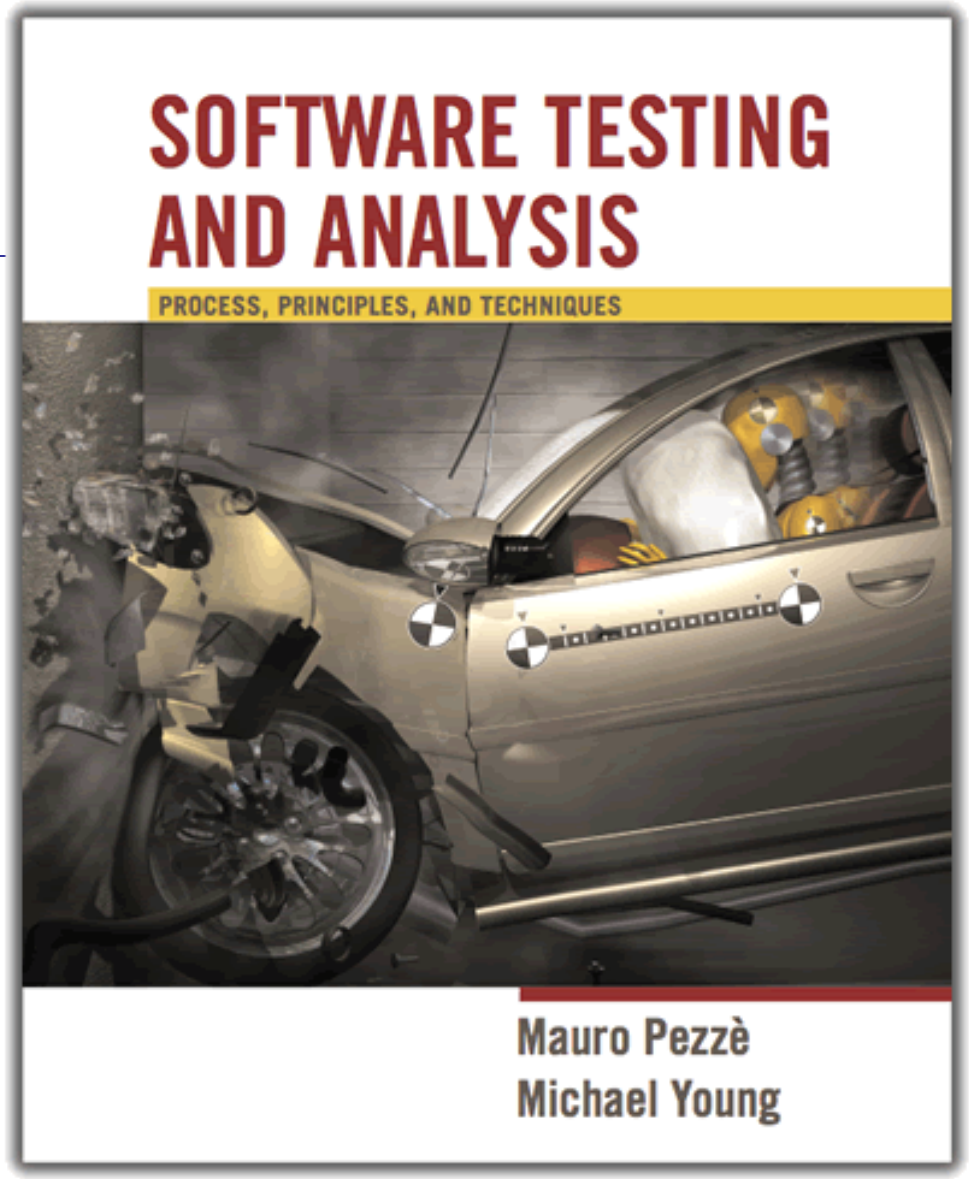
V Model of Testing



QUESTION: WHEN SHOULD YOU CONDUCT TESTS?

What you need for COMP30022

- Test Plan
- Acceptance criteria – guide test cases
- Test cases
- Test reports

Acknowledgements

- Slides from Dr. Chris Ewin, Prof Leon Sterling, Dr Bach Le, Mingye Li, and others
- https://cis-projects.github.io/project_based_course_notes/topics/testing.html